As per UGC guidelines an electronic bar code is provided to secure your paper

International Journal for Modern Trends in Science and Technology
Volume 12, Issue 04, pages 190-197.
ISSN: 2455-3778 online
Available online at: http://www.ijmtst.com/vol12issue04.html
DOI: https://doi.org/10.5281/zenodo.19324579

Deep Learning – Driven Visual Search and Image Recommendation

A.Venka Reddy, M.Sree Varsha, P.Usha, G.Bhanu Prakash, I.Venkata Siva Sai Kartikeya Sujit, M.Vishnu Sai

Department of CSE-Artificial Intelligence, PBR Visvodaya Institute of Technology and Science, Kavali, Andhra Pradesh, India.

To Cite this Article
A.Venka Reddy, M.Sree Varsha, P.Usha, G.Bhanu Prakash, I.Venkata Siva Sai Kartikeya Sujit & M.Vishnu Sai (2026). Deep Learning – Driven Visual Search and Image Recommendation. International Journal for Modern Trends in Science and Technology, 12(04), 190-197. https://doi.org/10.5281/zenodo.19324579

Article Info
Received: 28 February 2026; Revised: 18 March 2026; Accepted: 22 March 2026.

Copyright © The Authors ; This is an open access article distributed under the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

KEYWORDS

Deep Learning; Visual Search; Image Recommendation; EfficientNet; Feature Extraction; Cosine Similarity

ABSTRACT

In recent years, visual search and image recommendation systems have become essential in many fields such as e-commerce, multimedia management, digital libraries, and intelligent surveillance. Traditional keyword-based search methods mainly depend on textual descriptions, which are often incomplete, inaccurate, or unavailable. To overcome these limitations, this project presents a Deep Learning–Driven Visual Search and Image Recommendation system that focuses on visual content instead of text-based metadata.The proposed approach employs a pre-trained EfficientNetB4 convolutional neural network to extract meaningful visual features from images. Cosine similarity is then applied to compare these features and rank visually similar images from the dataset. A feature database is created to support efficient retrieval, and a Flask-based web interface is developed to enable real-time user interaction.

INTRODUCTION

The increasing use of digital platforms has resulted in the creation and sharing of a huge number of images every day. Online shopping websites, social media applications, medical systems, and multimedia platforms continuously generate visual data that must be stored, organized, and accessed efficiently. Because of this rapid growth, users now require intelligent systems that can quickly search and recommend images based on visual similarity rather than simple text descriptions.

Earlier image retrieval systems mainly depended on manually written keywords and tags. In many cases, these descriptions were incomplete, inaccurate, or totally missing. As a result, users often faced difficulties in finding relevant images using traditional keyword-based search methods. To solve this problem, content-based image retrieval techniques were introduced, which focus on analyzing the actual visual

properties of images instead of relying on external text information.

Image Retrieval Using Deep Learning

Visual search and image recommendation systems are now widely used in many real-world applications. Online marketplaces use them to suggest similar products, media platforms use them to organize content, and digital libraries use them to manage large collections. These systems aim to retrieve visually related images or provide suitable recommendations based on user input. However, as image datasets become larger and more diverse, traditional retrieval approaches face serious challenges related to speed and accuracy.

In the early stages of image retrieval research, systems mainly used handcrafted features such as color distributions, texture patterns, and shape descriptors. These features were manually designed using basic image processing techniques. Although such methods produced acceptable results for small datasets, they were not capable of capturing deeper semantic relationships between images. In addition, their performance was highly affected by changes in lighting conditions, camera angles, and background environments.

To improve retrieval quality, researchers later applied machine learning algorithms such as classification and clustering techniques. Methods including Support Vector Machines and k-Nearest Neighbors were used to group and compare images. However, since these approaches still relied on manually extracted features, their ability to adapt to new data and complex environments remained limited

The introduction of deep learning brought a major change to visual search and recommendation systems. Convolutional Neural Networks made it possible to learn multiple levels of visual representation directly from image pixels. These networks gradually learn from simple edges to complex object patterns, allowing systems to understand images in a more meaningful way. This capability significantly improved retrieval accuracy and system reliability.

2. LITERATURE SURVEY

The development of visual search and image recommendation systems has evolved gradually with the growth of digital image collections. In the beginning, researchers focused on extracting simple visual details such as color distribution, texture variation, and shape information. These features were manually designed using image processing techniques. While this method worked for controlled datasets, it was not reliable in real-world environments where lighting, background, and object orientation frequently change.

To improve matching accuracy, machine learning algorithms were later introduced. Techniques such as classification and clustering were used to group images with similar characteristics. Although these methods improved organization and comparison, they still relied on manually engineered features. As image databases increased in size, this approach became less practical because it required continuous manual tuning.

A major improvement occurred with the introduction of deep learning models. Instead of defining features manually, neural networks learned image representations automatically from raw pixel data. Convolutional Neural Networks proved especially effective because they capture both simple patterns and complex object structures. This automatic learning capability significantly enhanced the performance of visual retrieval systems.

Researchers then explored the use of pre-trained models to reduce development time. Models trained on large image datasets provided strong feature representations that could be reused for different applications. By applying transfer learning, these models could be adapted to new tasks without training from scratch.

In practical implementations, images are transformed into numerical representations that describe their visual properties. These representations are stored and compared using similarity calculations. To manage large-scale datasets efficiently, optimization techniques such as indexing and hashing were introduced. These techniques reduce search time and storage requirements.

Image recommendation systems extended retrieval techniques by incorporating user behavior. Early recommendation approaches mainly analyzed user interaction patterns or image metadata. However, these methods faced limitations when user data was limited or when new items were introduced.

Deep learning-based recommendation systems addressed these challenges by combining visual features with user information. Neural models learn relationships between images and user preferences, which improves personalization. Hybrid frameworks that integrate multiple data sources have demonstrated better accuracy in real-world applications.

Recent studies have also examined similarity learning models that directly learn distance relationships between images. Networks such as Siamese architectures help systems distinguish similar and dissimilar content more effectively. Additionally, lightweight deep models and optimization strategies have been developed to support real-time applications and large-scale deployment.

Limitations of Existing Approaches

Despite continuous improvements, several challenges remain:

- High computational requirements during training and deployment
- Dependence on large labeled datasets
- Limited generalization to unfamiliar image categories
- Performance issues with extremely large databases
- Latency concerns in real-time applications

3. PROPOSED SYSTEM

The proposed system is designed to perform efficient visual search and intelligent image recommendation by utilizing deep learning–based feature representations. In this approach, each image in the dataset is processed using a convolutional neural network to extract meaningful visual patterns.

All extracted features are stored in a feature database to enable fast and efficient retrieval. When a user uploads a query image through the Flask-based web interface, the system preprocesses the image and extracts its features using the same deep learning model. Cosine similarity is then applied to compare the query features with the stored dataset features.

The EfficientNetB4 model is adopted as the backbone architecture due to its balanced performance in terms of accuracy and computational efficiency. The final classification layers of the network are removed, allowing the model to function as a powerful feature extractor.

After processing, every image is transformed into a fixed-length numerical vector that represents its visual characteristics. These feature vectors capture important information such as texture, color distribution, shapes, and structural details. All generated vectors are then stored in a dedicated feature database for efficient access and retrieval.

When a user submits a query image, it undergoes the same preprocessing and feature extraction steps to ensure consistency. The extracted query feature vector is compared with the stored database vectors using cosine similarity, which measures the degree of similarity between two feature representations. Based on the similarity scores, the system ranks the dataset images and retrieves the top matching results.

Finally, the most relevant images are presented to the user as recommendations. This deep learning–driven approach improves retrieval accuracy, reduces manual dependency, and enables fast searching even in large-scale image collections. The proposed system is suitable for applications such as product search, medical image analysis, digital libraries, and multimedia content management.

3.1 Overall Workflow

Image Dataset Collection

↓

Image Preprocessing (Resize, Normalize, Augmentation)

↓

Deep Learning Feature Extraction

↓

Feature Database Storage

↓

Query Image Input (User Upload)

↓

Query Preprocessing

↓

Query Feature Extraction

↓

Similarity Matching (Cosine / Euclidean)

↓

Ranking of Similar Images

↓

Image Recommendation (Top-N Results)

3.2. Preprocessing and Standardization Module

This module prepares raw images for deep learning processing. All input images are resized to a fixed dimension and normalized to ensure uniformity. Noise removal and contrast adjustment are also applied to improve visual clarity.

Functions:

- Image resizing (e.g., 224 x 224 pixels)
- Pixel normalization
- Noise reduction
- Data augmentation (optional)

Formula (Normalization):

Inorm=I−IminImax−IminI_{norm} I_{min}}{I_{max} I_{min}}Inorm=Imax−IminI−Imin = \frac{I-

-

Where:

III = Original pixel value IminI_{min}Imin = Minimum pixel value

ImaxI_{max}Imax = Maximum pixel value

3.3. Deep Feature Extraction Module

`This module uses the EfficientNetB4 model to learn high-level visual representations from images. The final classification layers are removed so that the network works as a feature generator instead of a classifier.In this module, the EfficientNetB4 deep learning model is used to extract meaningful visual features from images. The final classification layer is removed, and feature vectors are generated for each image

Functions:

- Load pre-trained EfficientNetB4
- Remove output layer
- Extract feature vectors
- Convert images into numerical form

Formula (Feature Vector Representation):

F=[f1,f2,f3,...,fn]F = [f_1,f_2, f_3, ..., f_n]F=[f1,f2,f3,...,fn]

Where:

FFF = Feature vector

fnf_nfn = Individual feature values

3.4. Similarity Measurement Module

This module compares the query image features with stored dataset features. Cosine similarity is used to measure how close two images are in the feature space.

Functions:

- Receive query features
- Compare with database features
- Compute similarity score
- Select best matches

Formula (Cosine Similarity):

Similarity(A,B)=A·B ∣ ∣ A ∣ ∣ × ∣ ∣ B ∣ ∣ Similarity(A,B

) = \frac{A \cdot B}{||A|| \times ||B||}Similarity(A,B)= ∣ ∣ A ∣ ∣ × ∣ ∣ B ∣ ∣ A·B

Where: A,BA, BA,B = Feature vectors

∣∣A∣∣, ∣∣B∣∣ ∣ ∣A∣ ∣,∣ ∣B∣ ∣∣∣A∣∣, ∣∣B∣∣ ∣ ∣A∣ ∣,∣ ∣B∣ ∣ = Vector magnitudes

3.5. Ranking and Recommendation Module

This module arranges the retrieved images based on similarity scores and displays the most relevant results to the user. Higher similarity values indicate better matches.

Functions:

- Sort similarity scores
- Rank images
- Select Top-N results
- Display output

Formula (Ranking Score):

Rank=1N∑i=1NSiRank = \frac{1}{N} \sum_{i=1}^{N} S_iRank=N1i=1∑NSi

Where:

SiS_iSi = Similarity score

NNN = Number of retrieved images

Advantages of the Proposed System

The system provides accurate image retrieval using deep learning–based feature extraction.

It automatically learns visual features without manual intervention.

The search process is fast even for large image databases.

The system can be easily expanded with new images.

It works effectively under different lighting and background conditions.

Manual image tagging is minimized, reducing human effort.

Users can search using images instead of text queries.

It supports multiple application areas such as e-commerce and healthcare.

The use of EfficientNetB4 ensures balanced performance and resource usage.

The recommendation results are more relevant and reliable.

4. SYSTEM ARCHITECTURE

In this project, the system is organized in a step-by-step manner so that each operation is performed in a clear and efficient way. The main purpose of this architecture is to process images smoothly and provide accurate results within a short time. All major tasks such as image preparation, feature generation, comparison, and result display are connected in a single workflow.

At the beginning, the image dataset is collected and stored in the system. These images may be kept either on a local computer or on cloud storage, depending on availability. Before further processing, each image is standardized through basic operations such as resizing and normalization. This step is necessary to maintain uniformity among all input images.

After preprocessing, the images are passed to the deep learning model. In this project, EfficientNetB4 is used only for extracting important visual information. The classification part of the model is removed, and the remaining layers generate numerical values that represent the main visual characteristics of each image.

These numerical representations are saved in a separate storage area known as the feature database. This storage helps the system quickly access image information whenever a search request is received. Simple indexing techniques are applied to improve retrieval speed.

When a user uploads a query image, it follows the same processing steps as the dataset images. First, it is resized and normalized, and then its features are extracted using the same deep learning model.

This ensures that comparison is performed under identical conditions.

Next, the extracted features of the query image are compared with stored features using cosine similarity. This calculation determines how close two images are in terms of visual appearance. Images with higher similarity values are considered more relevant.



5. IMPLEMENTATION DETAILS

Step 1: Image Dataset Organization

Image samples were collected from trusted sources and arranged in a structured manner. Duplicate files, damaged images, and irrelevant data were removed to ensure high-quality input for system processing.

Step 2: System Environment Configuration

The required software libraries, including TensorFlow, Keras, NumPy, and OpenCV, were installed and verified. System settings were adjusted to support smooth execution of deep learning operations.

Step 3: Image Preparation Process

All images were resized according to the model's input requirements and pixel values were standardized. This process helped in maintaining uniformity and improving feature learning efficiency.

Step 4: Deep Learning Model Setup

The pre-trained EfficientNetB4 network was loaded into the system. Its classification component was removed so that the model could function exclusively as a feature extraction unit.

Step 5: Feature Vector Generation and Storage

Visual features were extracted from each dataset image and converted into numerical vectors. These vectors were stored in a structured database along with corresponding image references.

Step 6: Query Image Processing

User-submitted images were accepted through the interface and processed using the same preparation and feature extraction pipeline applied to the dataset images.

Step 7: Similarity Score Computation

The extracted query features were compared with stored feature vectors using cosine similarity. This computation produced relevance scores for identifying closely related images.

Step 8: Result Ranking and Visualization

Images were arranged based on their similarity scores, and the most relevant results were displayed through an interactive and user-friendly interface.

6. EXPERIMENTAL RESULTS

The performance of the proposed deep learning–based visual search system was evaluated using a dataset containing nearly 1,000 images classified into 10 different categories. Before processing, all images were resized to 224 × 224 pixels to meet the input requirements of the model. Each image was then converted into a 1,792-dimensional numerical feature vector using the EfficientNetB4 network.

6.1 Dataset and Experimental Setup

The experimental evaluation was performed using a dataset of approximately 1,000 images distributed across 10 categories. All images were standardized to a resolution of 224 × 224 pixels and processed through the EfficientNetB4 model to generate feature vectors of length 1,792. The system was implemented using Python and TensorFlow and executed on a standard computing platform.

6.2 Performance Evaluation Metrics

System efficiency and retrieval quality were assessed using commonly accepted performance measures, including precision, recall, and Top-5 accuracy. Cosine similarity was employed to calculate similarity scores between the query image and the images stored in the database. These evaluation parameters provided a clear understanding of the system's ability to retrieve relevant results.

6.3 Retrieval Accuracy Analysis

The experimental analysis demonstrated that the proposed system maintained high retrieval accuracy. The average precision was recorded at nearly 91%, while the recall rate reached approximately 88%. In addition, the Top-5 accuracy of about 93% indicated that most relevant images were positioned within the top search results. These findings highlight the effectiveness of deep feature extraction in representing visual information.

6.4 Response Time and Efficiency

The average processing time required to handle a single query image was measured at approximately 0.42 seconds. This result confirms that the system is capable of delivering fast responses even when operating on large image datasets. Furthermore, the optimized feature storage and retrieval mechanisms contributed significantly to improving overall search efficiency.

6.5 Comparative Performance Study

A comparative analysis was conducted between the proposed deep learning approach and traditional handcrafted feature-based methods. The results showed that the proposed system achieved better accuracy and stability. It maintained consistent performance under variations in lighting conditions, background complexity, and object orientation. In addition, the use of EfficientNetB4 provided a balanced combination of computational efficiency and retrieval performance.

7. DISCUSSION

This project involved designing and testing a visual search and image recommendation system using deep learning techniques. Throughout the development

process, different stages of the system were analyzed to understand their impact on accuracy, speed, and usability. The overall performance showed that the chosen approach was effective for handling image-based search tasks.

One of the main observations made during this work was the importance of proper image preparation. When images were resized and normalized correctly, the system produced more consistent results. Applying the same preprocessing steps to both stored images and user input images helped in reducing unnecessary variations and improved comparison accuracy.

The feature extraction stage played a major role in determining system performance. By using the EfficientNetB4 model, important visual information such as shape patterns, texture details, and structural features was captured effectively. These extracted features allowed the system to clearly differentiate between similar and unrelated images. The use of a pre-trained model also reduced development time and simplified implementation.

8. CONCLUSION

This project focused on the development of a Deep Learning–Driven Visual Search and Image Recommendation system for retrieving images based on visual similarity. The main goal was to reduce the limitations of traditional text-based and manually designed feature methods by applying deep learning techniques for automatic feature extraction and comparison.

The system used EfficientNetB4 as the core model for generating visual representations. By removing the classification layers, the network was adapted to function as a feature extraction tool. This allowed the system to convert images into compact numerical vectors that describe important visual characteristics.

These feature vectors captured information related to shape, texture, color patterns, and spatial structure. Using this representation, the system was able to compare images more accurately and reliably. Experimental results showed that the proposed approach produced better retrieval performance when compared to conventional techniques.

9. FUTURE SCOPE

Although the proposed system achieved satisfactory performance, several improvements can be considered for future development. These enhancements can further increase system accuracy, scalability, and user satisfaction.

One important direction for future work is the use of larger and more diverse datasets. Including images from different domains, environments, and real-life scenarios can help the model learn more generalized features. This will improve system reliability under varying lighting conditions, backgrounds, and object appearances.

Another potential improvement involves the adoption of advanced indexing and search techniques. The integration of high-performance libraries such as FAISS or approximate nearest neighbor methods can significantly reduce search time when dealing with very large databases. This will make the system more suitable for large-scale applications.

Conflict of interest statement

Authors declare that they do not have any conflict of interest.

REFERENCES

[1] Li, X., Zhang, Y., & Wang, J. (2021). Deep learning techniques for large-scale image retrieval: A review. Pattern Recognition Letters, 146, 45–56.

[2] Chen, Y., Liu, H., & Zhao, Q. (2021). Visual search based on deep feature representation and similarity learning. Journal of Visual Communication and Image Representation, 78, 103198.

[3] Wang, S., Zhou, Z., & Liu, Y. (2022). Efficient image retrieval using convolutional neural networks and feature indexing. Multimedia Tools and Applications, 81(5), 6891–6907.

[4] Kumar, R., Patel, A., & Singh, P. (2022). Content-based image retrieval using deep neural networks. International Journal of Computer Applications, 184(21), 12–18.

[5] Zhang, H., Wu, X., & Chen, L. (2023). Deep feature learning for visual search and recommendation systems. IEEE Access, 11, 45732–45744.

[6] Rahman, M., Islam, T., & Hossain, M. (2023). An intelligent image recommendation framework using deep learning. Expert Systems with Applications, 213, 118956.

[7] Liu, J., Yang, W., & Sun, K. (2024). Scalable visual search system using deep convolutional networks. Information Sciences, 650, 119–132.

[8] Sharma, V., Mehta, N., & Verma, R. (2024) Performance analysis of deep learning models for image retrieval. Journal of King Saud University – Computer and Information Sciences, 36(2), 101543.

[9] Park, S., Kim, J., & Lee, H. (2025). Multimodal deep learning for image retrieval and recommendation. IEEE Transactions on Multimedia, 27, 345–358.

[10] Das, A., Roy, S., & Banerjee, D. (2025). A hybrid deep learning framework for visual search applications. ACM Transactions on Multimedia Computing, Communications, and Applications, 21(1), 1–19.
[11] Zhou, Y., Chen, M., & Li, X. (2021). Deep feature learning for content-based image retrieval systems. Neurocomputing, 448, 189–201.
[12] Patel, R., Shah, K., & Desai, M. (2021). Image retrieval using convolutional neural networks and similarity metrics. Procedia Computer Science, 192, 456–465.
[13] Singh, A., Kumar, V., & Gupta, R. (2022). A comparative study of deep learning models for visual search. International Journal of Information Technology, 14(3), 1345–1353.
[14] Huang, Z., Wang, L., & Xu, Q. (2022). Learning discriminative embeddings for large-scale image retrieval. IEEE Access, 10, 67234–67245.

International Journal for Modern Trends in Science and Technology
IJMTST